# Operating Systems Assignment 2 – Task 3 (Q3)

**Threads – Traffic-Camera Number-Plate Reader** (CO2, K5 – Evaluate, 6 marks). Environment: Ubuntu 24.04.5 LTS, gcc 13.3.0, Intel Xeon @ 2.1 GHz with **2 CPUs**. Compile: `gcc -std=c11 -O1 -Wall -Wextra -pthread`. Structure follows the expected flow *Model* → *Design* → *Implement* → *Measure* → *Compare* → *Evaluate* → *Recommend*.

## 1. Model

64 cameras each deliver a frame that must be processed (plate recognition). The first version handles cameras one after another, so total time = 64 × per-frame time and falls behind at peak. Frames are independent of each other, so the work is parallelisable; the only shared items are the work index and the result counter. I model recognition as a fixed CPU loop per frame (≈ 7.8 ms) and a second workload that adds a 20 ms wait for the frame to arrive over the network, to represent I/O.

## 2. Which resources threads share and which they own

| Shared by all threads of a process | Private to each thread |
|---|---|
| Code (text) segment, global/static data, heap | Thread ID and its own stack (local variables, call frames) |
| Open files, sockets, file-descriptor table | CPU register set including the program counter and stack pointer |
| Signal handlers, working directory, process ID, user/group IDs | Thread-local storage (`__thread`), errno, signal mask, scheduling priority/state |

Sharing is why thread creation and switching are cheap (no new address space) and why shared data such as the plate counter need locking.

## 3. User-level vs kernel-level threads, and the mapping models

| | User-level threads | Kernel-level threads |
|---|---|---|
| Managed by | Thread library in user space; kernel sees one process | Operating-system kernel |
| Create/switch cost | Very low (no system call) | Higher (system call, kernel scheduling), still far cheaper than a process |
| Blocking I/O | One blocking call blocks all threads of the process | Only the calling thread blocks; others keep running |
| Multicore | Cannot run in parallel on several cores | Can run truly in parallel on several cores |

| Model | Mapping | Advantage | Drawback |
|---|---|---|---|
| Many-to-one | Many user threads → 1 kernel thread | Cheap management | No parallelism; one blocking call stops all |
| One-to-one | Each user thread → 1 kernel thread | True parallelism, blocking does not stop others | Each thread costs a kernel thread; limit on count |
| Many-to-many | M user threads → N ≤ M kernel threads | Parallelism with a bounded kernel-thread count | Complex to implement; rarely used now |

**Linux POSIX threads (NPTL) use one-to-one.** For this problem that is the right model: camera threads blocked on the network must not block the others, and CPU-heavy recognition must be able to use every core.

## 4. Implement: multithreaded program

`plates.c` runs the 64 camera frames in three modes: sequential, **threads** (a pool of 4/8/16 workers pulling the next camera index under a mutex, or 64 = one thread per camera) and **processes** (alternative in Section 8, results combined through a pipe). Usage: `./plates thr 8 cpu`.

```
/* Task 3 - Number-plate reader: 64 cameras, POSIX threads vs processes.
   Compile: gcc -std=c11 -O1 -Wall -Wextra -pthread -o plates plates.c
   Usage:   ./plates &lt;mode&gt; &lt;workers&gt; &lt;workload&gt;
            mode: seq | thr | proc   workload: cpu | mixed   */
#define _GNU_SOURCE
```

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;unistd.h&gt;
#include &lt;pthread.h&gt;
#include &lt;sys/wait.h&gt;
#include &lt;time.h&gt;
#define CAMERAS 64
#define WORK 6000000UL          /* "image processing" loop per camera */
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&amp;t);return t.tv_sec+t.tv_nsec*1e-9;}
static int mixed = 0;
static volatile unsigned long sink;
static unsigned long process_frame(int cam){      /* CPU work (+ optional I/O wait: camera frame fetch) */
    if (mixed) usleep(20000);                      /* 20 ms waiting for the frame over the network */
    unsigned long h = 14695981039346656037UL ^ cam;
    for (unsigned long i = 0; i &lt; WORK; i++) h = (h ^ i) * 1099511628211UL;
    return h;
}
static int next_cam = 0;                           /* shared work counter */
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static long plates_ok = 0;                         /* shared result counter (protected) */
static void *worker(void *a){
    (void)a;
    for(;;){
        pthread_mutex_lock(&amp;m); int c = next_cam++; pthread_mutex_unlock(&amp;m);
        if (c &gt;= CAMERAS) break;
        unsigned long h = process_frame(c); sink = h;
        pthread_mutex_lock(&amp;m); plates_ok++; pthread_mutex_unlock(&amp;m);
    }
    return NULL;
}
static void *one_cam(void *a){ int c=(int)(long)a; sink=process_frame(c);
    pthread_mutex_lock(&amp;m); plates_ok++; pthread_mutex_unlock(&amp;m); return NULL; }
int main(int argc,char**argv){
    const char*mode = argc&gt;1?argv[1]:"seq"; int w = argc&gt;2?atoi(argv[2]):1;
    mixed = (argc&gt;3 &amp;&amp; !strcmp(argv[3],"mixed"));
    double t0=now();
    if(!strcmp(mode,"seq")){ for(int c=0;c&lt;CAMERAS;c++){sink=process_frame(c);plates_ok++;} }
    else if(!strcmp(mode,"thr")){
        pthread_t th[CAMERAS];
        if(w&gt;=CAMERAS){ for(long c=0;c&lt;CAMERAS;c++) pthread_create(&amp;th[c],NULL,one_cam,(void*)c);
                        for(int c=0;c&lt;CAMERAS;c++) pthread_join(th[c],NULL); }
        else { for(int i=0;i&lt;w;i++) pthread_create(&amp;th[i],NULL,worker,NULL);
               for(int i=0;i&lt;w;i++) pthread_join(th[i],NULL); }
    } else {                                       /* processes: results combined through a pipe */
        int fd[2]; pipe(fd);
        for(int i=0;i&lt;w;i++) if(fork()==0){
            long cnt=0; for(int c=i;c&lt;CAMERAS;c+=w){ sink=process_frame(c); cnt++; }
            write(fd[1],&amp;cnt,sizeof cnt); _exit(0); }
        for(int i=0;i&lt;w;i++){ long cnt; read(fd[0],&amp;cnt,sizeof cnt); plates_ok+=cnt; }
        while(wait(NULL)&gt;0);
    }
    printf("%-4s workers=%-2d %-5s frames=%ld time=%.3f s\n",mode,w,mixed?"mixed":"cpu",plates_ok,now()-t0);
    return 0;
}
```

### *Sample execution*

```
$ ./plates seq 1 cpu
seq  workers=1  cpu   frames=64 time=0.501 s
$ ./plates thr 8 cpu
thr  workers=8  cpu   frames=64 time=0.253 s
$ ./plates thr 64 mixed
thr  workers=64 mixed frames=64 time=0.273 s
```

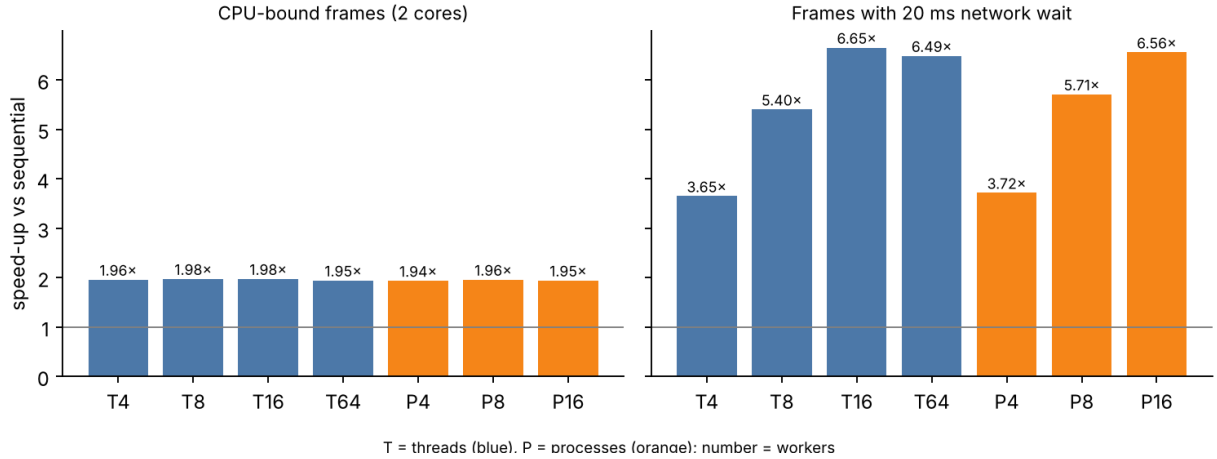
All 64 frames are always processed (frames=64), so the parallel runs produce the same result as the sequential one.

## 5. Measure: speed-up for each pool size

Each value is the **median of 5 runs** (`bench.sh`). Speed-up $S = T_{seq} / T_{parallel}$; efficiency = S ÷ number of cores.

| Configuration | CPU-bound time (s) | Speed-up | Efficiency (÷2 cores) | With network wait: time (s) | Speed-up |
|---|---|---|---|---|---|
| Sequential (1 thread) | 0.496 | 1.00× | 50% | 1.810 | 1.00× |
| Threads, pool of 4 | 0.253 | 1.96× | 98% | 0.496 | 3.65× |

| Configuration | CPU-bound time (s) | Speed-up | Efficiency (÷2 cores) | With network wait: time (s) | Speed-up |
|---|---|---|---|---|---|
| Threads, pool of 8 | 0.251 | 1.98× | 99% | 0.335 | 5.40× |
| Threads, pool of 16 | 0.251 | 1.98× | 99% | 0.272 | 6.65× |
| Threads, pool of 64 (one per camera) | 0.255 | 1.95× | 97% | 0.279 | 6.49× |
| Processes, 4 | 0.256 | 1.94× | 97% | 0.487 | 3.72× |
| Processes, 8 | 0.253 | 1.96× | 98% | 0.317 | 5.71× |
| Processes, 16 | 0.255 | 1.95× | 97% | 0.276 | 6.56× |




## 6. Evaluate

### *6.1 Why the speed-up stops increasing*

**CPU-bound workload:** 4 threads give 1.96×, and 8, 16 and 64 threads give the same ≈ 1.95–1.98×. The machine has only **2 cores**, so at most 2 threads can run at the same instant; extra threads just wait for a core, and more threads add creation and switching cost without adding computing power. The ceiling is the number of cores (speed-up ≤ p cores), and by Amdahl's law $S \le 1/((1-f) + f/p)$, where the serial fraction (1–f) — creating threads, taking the work-index mutex, joining — is tiny here, so we reach ≈ 98% of the 2× limit.

**Workload with network wait:** while a thread waits for its frame, the core is free for another thread, so more threads than cores *do* help: 3.65× (4 threads) → 5.4× (8) → 6.65× (16). It then stops growing, and one thread per camera (64) is slightly slower (6.49×) than 16: the total CPU work (≈ 0.5 s over 2 cores = 0.25 s) is a floor that waiting cannot hide, and 64 threads add creation, stack memory and scheduling overhead. Rule of thumb: threads ≈ cores × (1 + wait time ÷ compute time) = 2 × (1 + 20/7.8) ≈ 7 for balance, and measurement shows gains continuing up to 16 as the ratio varies.

### *6.2 What happens to shared counters without protection*

`race.c`: two threads each add 1 to a shared counter 1,000,000 times.

```
expected      : 2000000
unprotected   : 1951038  (lost 48962 updates)
mutex         : 2000000
atomic        : 2000000
```

**Explanation:** `counter++` is three steps — load, add, store. When two threads interleave (both load 5, both store 6) one update is lost. This is a **race condition**: the result depends on timing and changes from run to run (here 48,962 updates lost, ≈ 2.4%). A mutex (or an atomic increment) makes the update indivisible and gives the exact 2,000,000. The cost: the mutex serialises that tiny section, so I keep the critical section short (only the index fetch and counter increment, never the image processing).

### *6.3 Evaluation of the thread models for this system*

With 1:1 kernel threads, both workloads gain from threads. Many-to-one would remain at ≈ 1× on a multicore machine for the CPU work (no parallelism) and a blocking frame read would freeze all cameras. Many-to-many

would give no extra benefit over a bounded pool at 1:1 for 64 cameras.

## 7. Design decision

**Use a bounded pool of kernel (1:1) threads, sized by workload: about the number of cores for pure CPU recognition, and about 2–4× the cores (8–16 on this machine) when frames come over the network.** Do not create one thread per camera as the default: 64 threads (6.49×) are no faster than 16 (6.65×) and cost more memory, and on a city system with thousands of cameras one-per-camera would not scale. Shared counters must be protected by a mutex or atomic operation.

## 8. Alternative: several processes instead of threads

**Solution:** `./plates proc N` forks N child processes; each processes cameras i, i+N, i+2N… and writes its frame count to a pipe; the parent reads N counts and adds them. No locking is needed because nothing is shared — results travel by IPC.

| Aspect | Threads | Processes |
|---|---|---|
| Speed-up measured (CPU / with wait) | 4: 1.96× / 3.65×; 8: 1.98× / 5.40×; 16: 1.98× / 6.65× | 4: 1.94× / 3.72×; 8: 1.96× / 5.71×; 16: 1.95× / 6.56× |
| Creation time (measured in Task 1, same machine) | ≈ 60 µs per pthread_create+join | ≈ 246 µs per fork+exit+wait (≈ 4× slower) |
| Switching time | ≈ 1.4 µs (same address space, no TLB flush on real hardware) | ≈ 1.3 µs measured; plus TLB flush/cache loss with a different address space |
| Memory | Shared code, heap and data; each thread adds only a stack (a few KB committed) | Each process has its own page tables and private data (copy-on-write at fork); more memory per worker |
| Combining results | Shared variable + mutex/atomic (fast, but race risk) | IPC: pipe/shared memory/socket (system calls, copying; no race on data) |
| Fault isolation | A crash kills all threads | A crash kills only that process |
| Time complexity | Create O(1) per thread | Create O(address-space size) for page-table copy |
| Space complexity | O(M + p·s), s = stack | O(p·M) |

**Finding:** for 64 frames of ~8 ms work, the measured end-to-end times of processes and threads are almost identical (e.g. 0.253 s vs 0.251 s CPU-bound with 8 workers), because creation cost (≈ 0.25 ms × a few workers) is tiny compared with 0.25 s of work. Processes would win when isolation matters (a bad frame crashing the recogniser must not stop other cameras) and threads win when creation is frequent, memory is tight or results must be shared at high rate. For long-running, small pools the two perform the same, so the choice should be driven by isolation vs sharing, not raw speed.

## Limitations

Measured on a 2-core virtualised cloud container; on an 8- or 16-core machine the CPU-bound speed-up would keep growing up to the core count, then flatten. Frame processing is a synthetic loop, not real plate recognition. Run the programs on your own machine, commit them with outputs (`Task03/plates.c, race.c, bench.sh`) and use your own timings, with your name, register number and GitHub link added.